Развој софтвера 2

Архитектура и модуларност

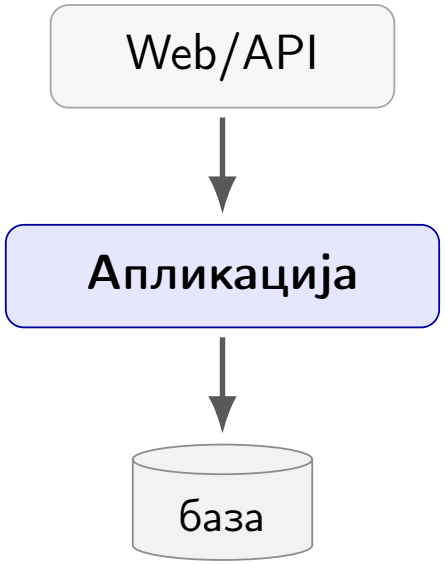
Стефан Капунац

Математички факултет, Универзитет у Београду

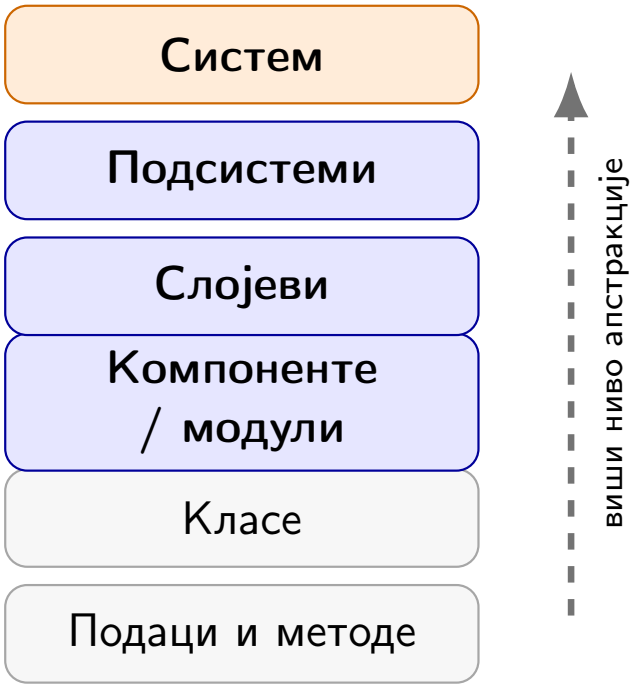
школска 2026/27. година

Где смо стигли?

- Прошли пут: једноставан систем расте и појављују се нови проблеми
- Данас: још увек **једна апликација**
- Питање: како је организовати тако да може да расте без хаоса?



Подижемо ниво апстракције



PC2: модули, слојеви, подсистеми, систем

PC1: углавном класе, објекти, методе

Веза са PC1

Исти принципи доброг дизајна сада се примењују на веће јединице: модуле, слојеве и подсистеме.

Модули и слојеви

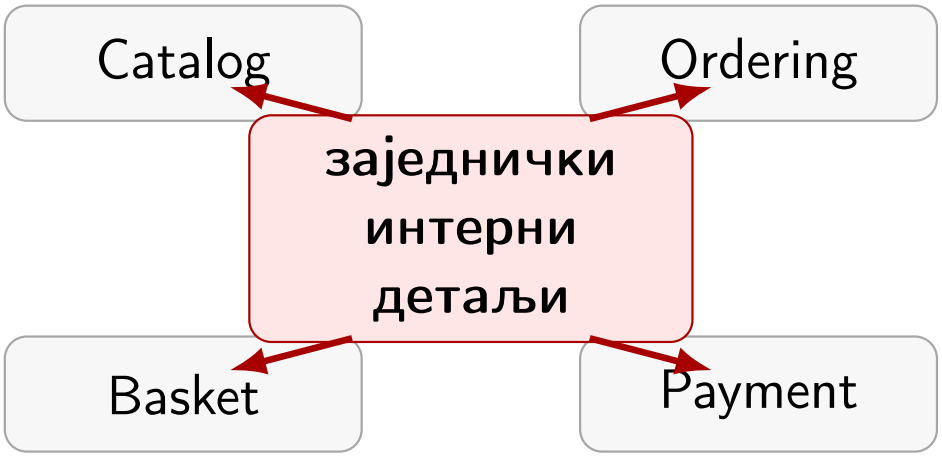
Модул није само фолдер

- Модул окупља једну логичку одговорност
- Излаже мали, јасан јавни интерфејс
- Скрива унутрашњу имплементацију и податке
- Други делови система зависе од уговора, не од детаља

Наставак идеје са PC1

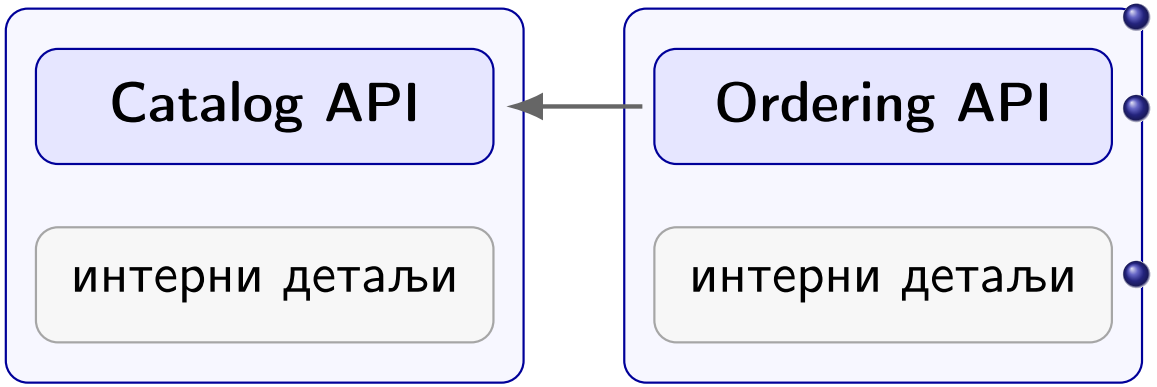
Не понављамо врсте кохезије и спрегнутости — исти принцип сада посматрамо на нивоу модула и целог система.

Када границе нису стварне



- сваки део зна интерне детаље осталих
- промена у једном делу се шири кроз систем
- називи фолдера не стварају модуларност

Граница постоји када постоји уговор



- јавни API модула
- интерни детаљи су сакривени
- зависности су видљиве и контролисане

Како организујемо код?

По техничком типу

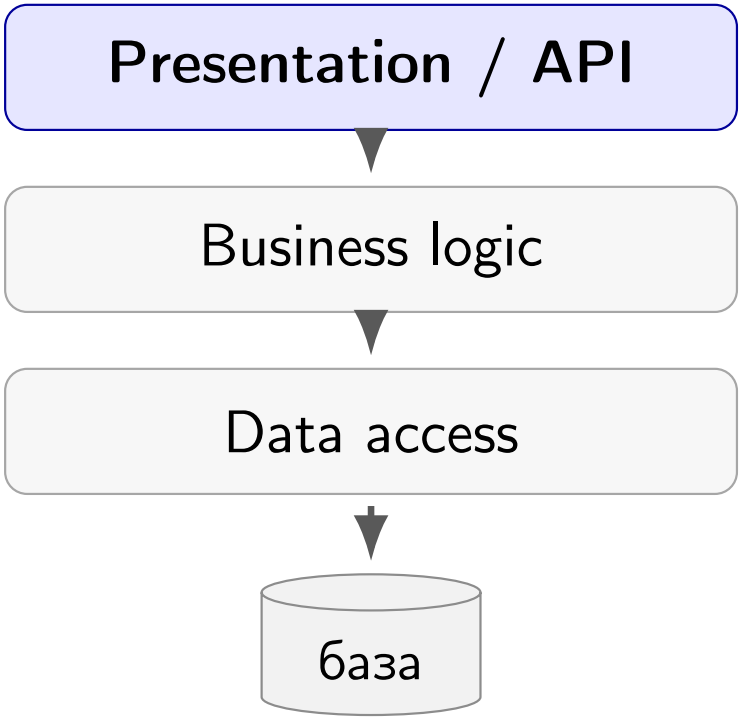
- Controllers
- Services
- Repositories
- Models

По функционалности

- **Catalog**
- **Basket**
- **Ordering**
- **Payment**

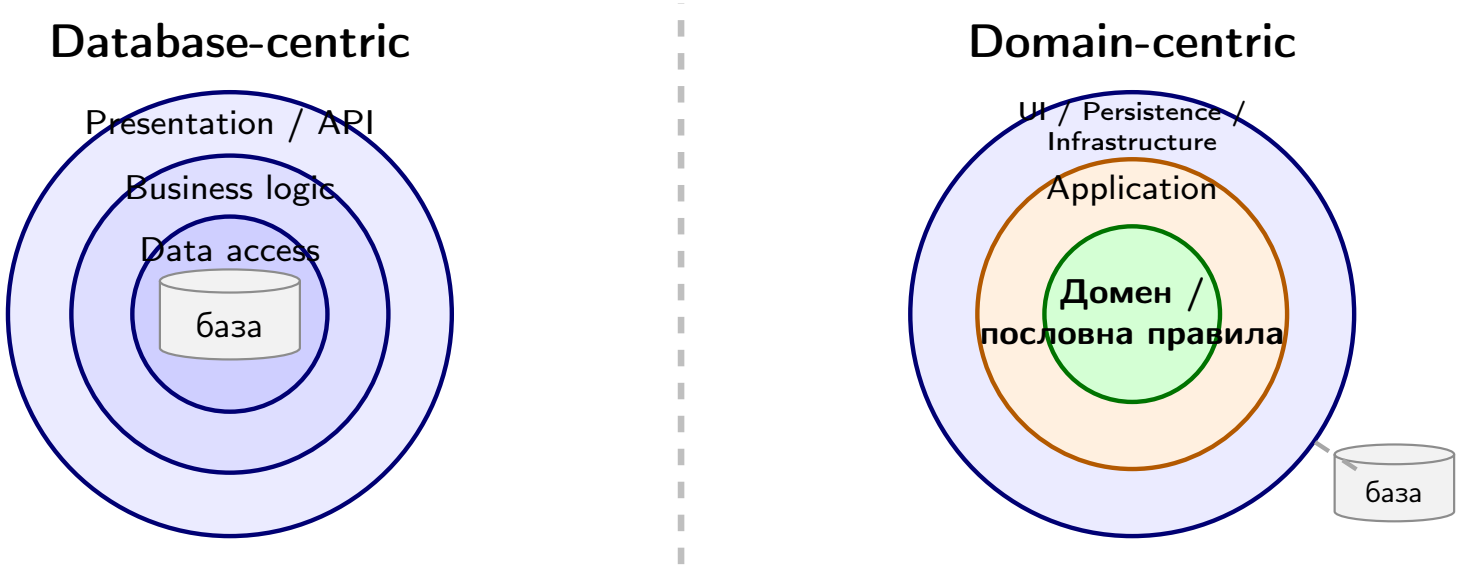
- Технички слојеви групишу сличне врсте компоненти
- Функционална организација групише код који заједно реализује једну пословну целину
- Ове идеје нису супротстављене: модул може имати сопствене унутрашње слојеве

Слојевита архитектура



- раздваја UI, пословну логику и приступ подацима
- једноставан и природан почетни модел
- сваки слој познаје ограничен број суседних слојева

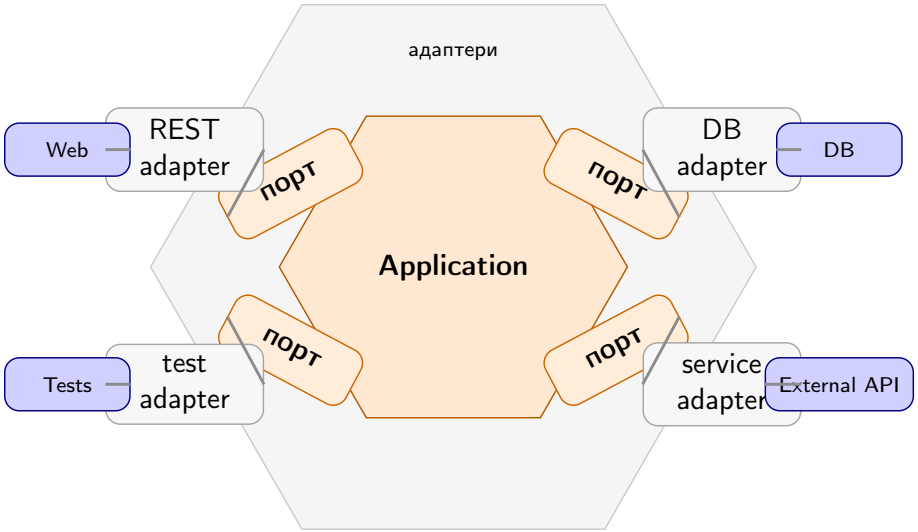
Шта стављамо у центар архитектуре?



- База и framework су важни, али су инфраструктурни детаљи
- Пословна правила треба да буду што мање везана за конкретну технологију

Зависности и границе

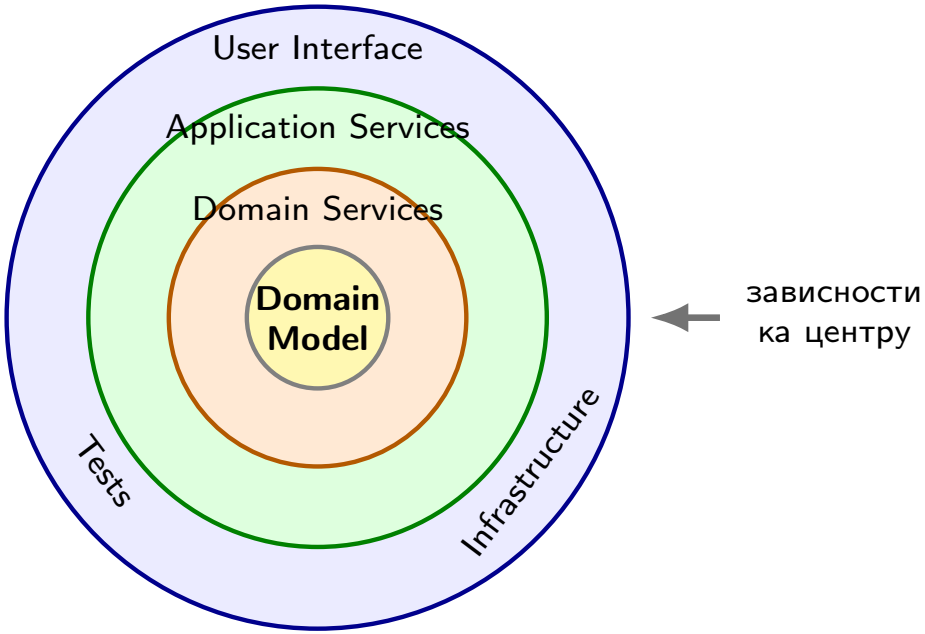
Hexagonal Architecture: Ports & Adapters



- језгро комуницира са спољним светом преко **портова**
- **адаптери** везују порт за конкретну технологију
- исти порт може имати више адаптера

Према оригиналном дијаграму Alistair Cockburn-а, *Hexagonal Architecture / Ports and Adapters*.

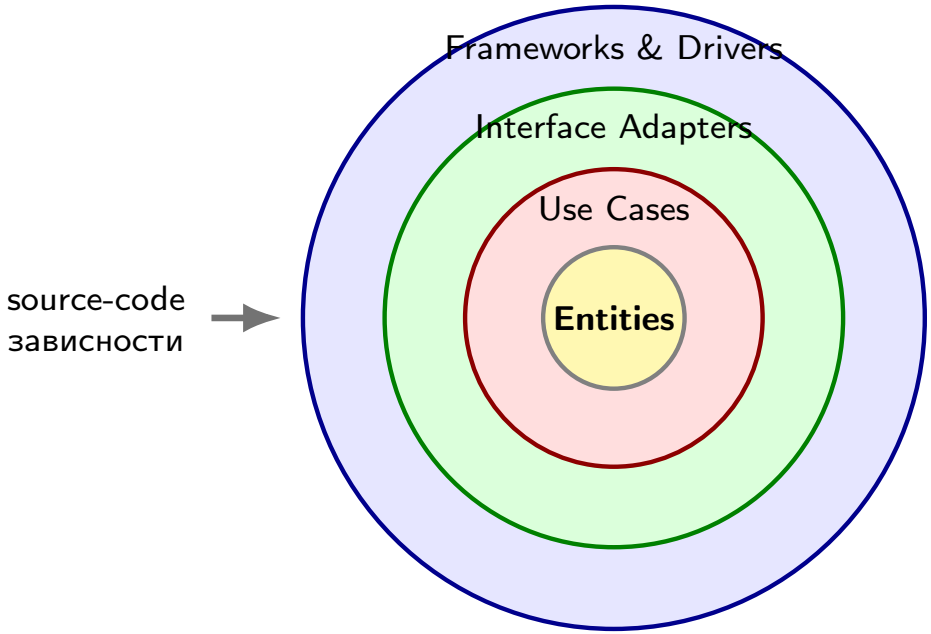
Onion Architecture



- домен је у **центру**
- зависности су усмерене ка центру
- инфраструктура је на спољној ивици

Према оригиналном дијаграму Jeffrey Palermo-a, *Onion Architecture* (2008).

Clean Architecture



- унутра су **business rules** и use cases
- framework, UI и база су спољни детаљи
- **Dependency Rule**: зависности иду ка унутра

Према оригиналном дијаграму Robert C. Martin-a, *The Clean Architecture* (2012).

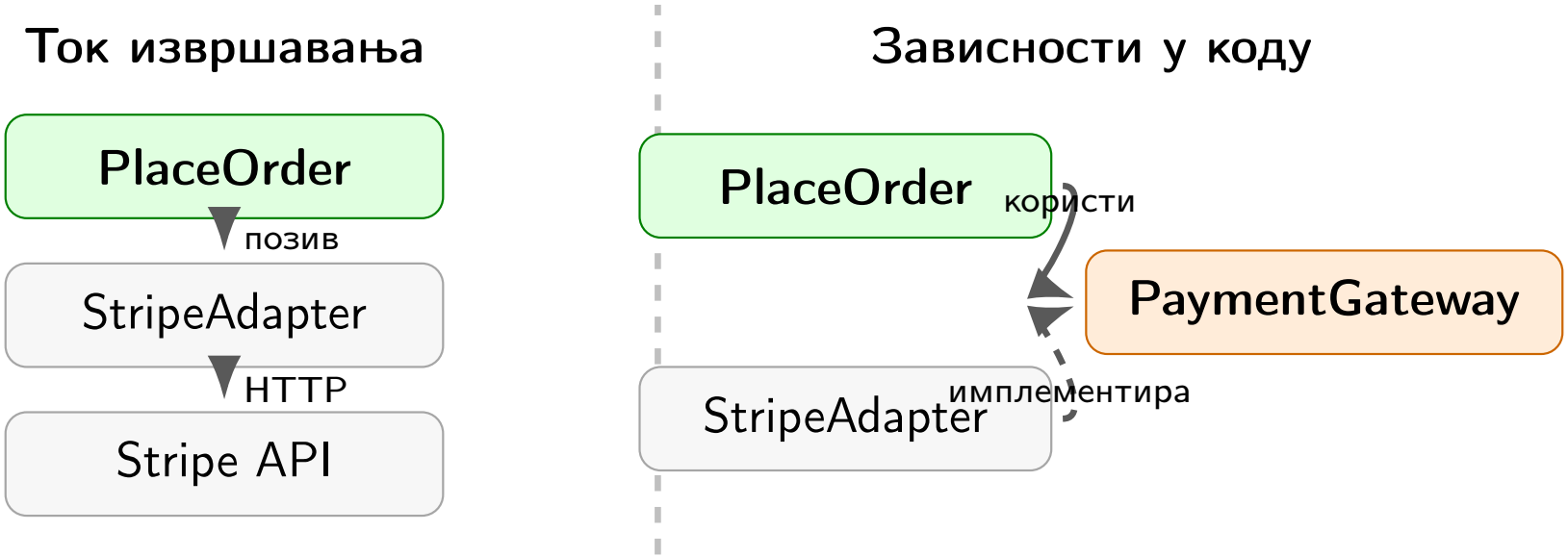
Различита имена, сличан принцип

- Пословна логика не треба да зависи од UI-ја, базе или framework-а
- Инфраструктурни детаљи остају на ивици система
- Зависности се усмеравају ка стабилнијем језгру и апстракцијама
- Таква организација олакшава замену технологија и тестирање језгра

Шта је важно за нас?

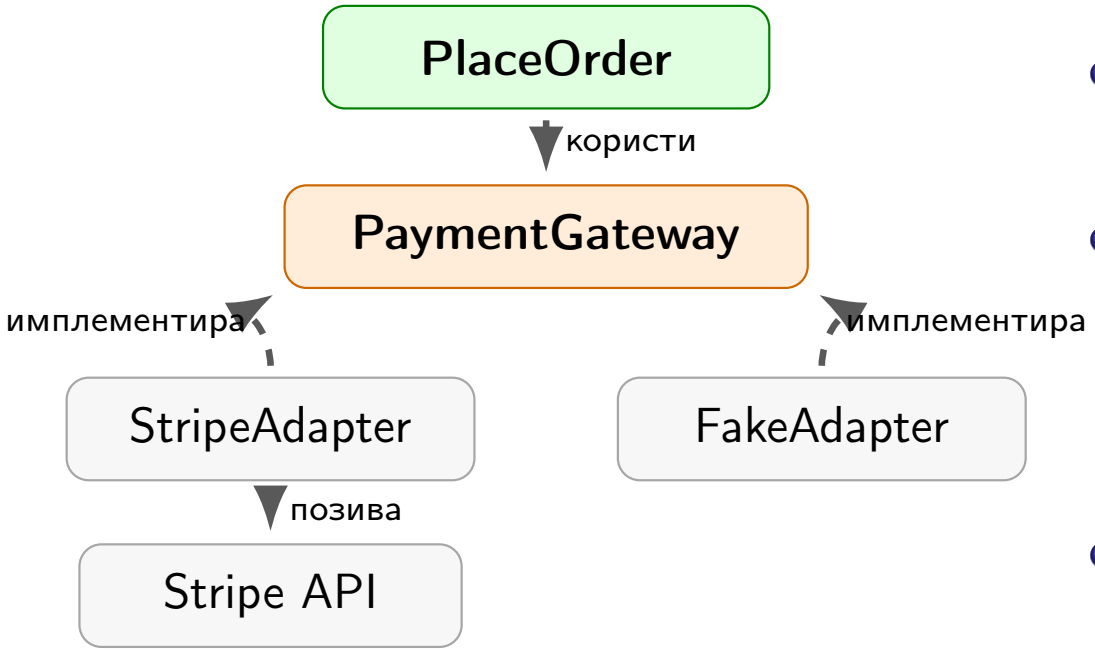
Не учимо дијаграме и називе слојева напамет — гледамо **ко од кога зависи** и шта је систем способан да промени без ланчане реакције.

Ток извршавања није исто што и зависност



- Ток извршавања на крају стиже до конкретног Stripe адаптера
- Пословна логика у коду зависи само од `PaymentGateway` уговора
- И конкретан адаптер зависи од тог уговора — не од пословне логике

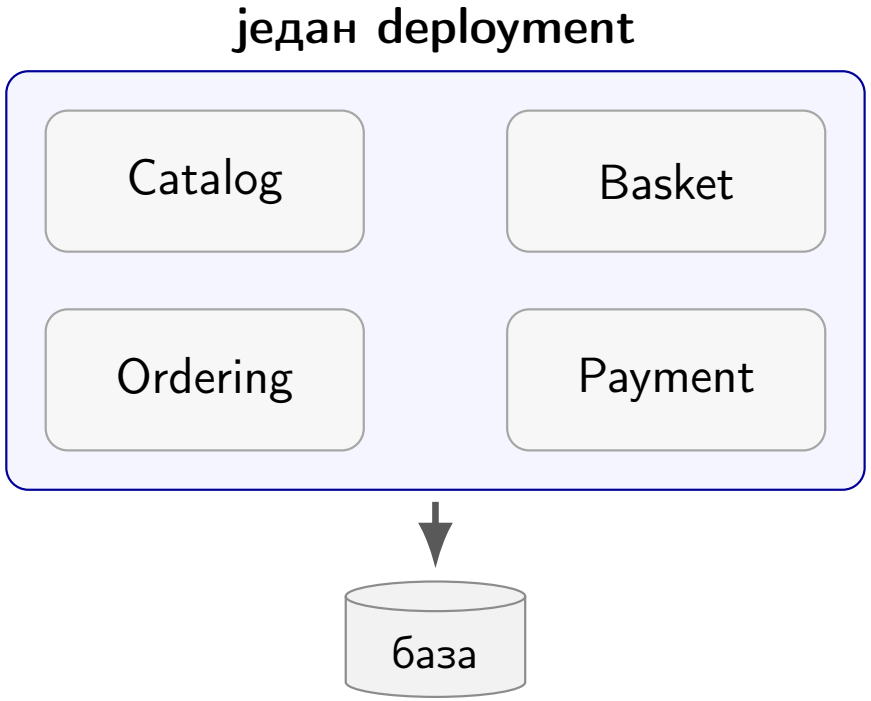
Конкретан пример: плаћање



- пословна логика тражи „наплати“
- не мора да зна да ли је Stripe, други provider или fake за тест
- технологија остаје на ивици система

Модуларни монолит

Модуларни монолит



- једна deployable апликација
- јасне границе модула по пословним целинама
- контролисане зависности
- подаци имају јасног власника

Шта значи „Catalog поседује своје податке“?

- Ordering не чита директно Catalog табеле
- Не мења Catalog податке „иза леђа“ модула
- До потребних података долази преко јавног уговора модула
- Физички база и даље може бити заједничка

Поента

Ownership је пре свега правило архитектуре — не захтева одмах посебан сервер базе.

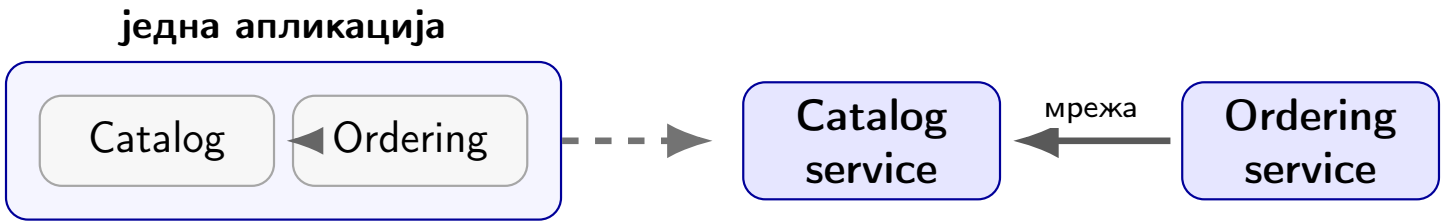
Зашто не бисмо одмах направили микросервисе?

- Позив између модула је локалан, брз и једноставан
- Једна трансакција може лакше обухватити више промена
- Deployment и локални развој су једноставнији
- Нема мрежних кварова између модула

Цена

Модули се и даље deploy-ују и скалирају као једна целина.

Од модула до сервиса



- Јасна граница модула може постати кандидат за границу сервиса
- Али физичко раздвајање доноси мрежу, partial failure и сложенији deployment

Када су Clean / Hexagonal идеје стварно корисне?

- Када имамо значајну пословну логику коју желимо да изолујемо
- Када база, спољни API-ји или framework-и не треба да диктирају домен
- Када желимо лакше тестирање језгра без инфраструктуре
- Када очекујемо да се спољни детаљи мењају

А када нису?

За малу CRUD апликацију додатни слојеви и интерфејси лако постану непотребна церемонија.

Шта треба да остане

1. Модуларност значи стварне границе и контролисане зависности, не само фолдере.
2. Clean / Hexagonal нам дају језик за одвајање пословног језгра од инфраструктуре.
3. Модуларни монолит задржава једноставан deployment, али уводи дисциплину граница.
4. Следеће питање је: када вреди те границе физички раздвојити у микросервисе?

За даље читање

- D. L. Parnas — *On the Criteria To Be Used in Decomposing Systems into Modules*
- Alistair Cockburn — *Hexagonal Architecture (Ports and Adapters)*
- Robert C. Martin — *The Clean Architecture*
- Microsoft .NET Architecture — *Common Web Application Architectures*